Obsidian Wikilinks and the Karpathy Wiki from the Ground Up

A hand-curated knowledge graph that expands an AI's working memory — with the Karpathy Wiki plugin as the local-first retriever

Christian Macion

2026-07-09 · public edition

Made by **Christian Macion** — Quantitative Researcher & AI Engineer. *For the practitioner who wants a personal knowledge graph that an AI can actually walk.*
Built in the …from the Ground Up mentee-edition format. Sources: `community.obsidian.md/plugins/karpathywiki`, `gist.github.com/karpathy/442a6bf555914893e9891c11519de94f`, `aclanthology.org/2024.tacl-1.9/`, `research.trychroma.com/context-rot`, `arxiv.org/abs/2510.05381`.
Numbers tagged SCHEMATIC are illustrative.

Part A. The shape of a knowledge graph

ORIENT: *the big picture before the detail*

Three names show up a lot in the next ten chapters: **Obsidian**, the editor; **wikilinks**, the syntax; and **the Karpathy Wiki plugin**, the LLM-backed retriever that sits on top. This Part is the shortest correct description of all three. If you read nothing else, read pages 1–4 of this Part.

BOTTOM LINE Obsidian is a **local-first markdown editor** with a **wiki-link graph** as its killer feature. The vault is just a folder of `.md` files; the graph is derived; Obsidian walks it for you.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- describe the three layers of an Obsidian vault (notes, graph, plugin)
- open a vault, see its graph, and read a wikilink in plain text
- explain why 'plain markdown, no database' is the design choice that makes everything else possible

Prerequisites: having used any plain-text editor (vim, VS Code, TextEdit).

ORIENT: *the big picture before the detail*

If you have ever wished a notes app would stop locking your data into a proprietary format, Obsidian is the answer. The folder is the vault. The files are markdown. There is *no* database, no sync server you must use, no account. Git-tracked, scp-able, `grep`-able. The cost of all that freedom is that you write the structure — Obsidian won't.

PRIME: *new terms, one line each*

- **vault**: a folder on disk; the unit Obsidian opens
- **wikilink**: [[note-id]] — six chars in a Markdown file that Obsidian renders as a link
- **graph**: derived from the wikilinks — every note a node, every [[link]] an edge
- **graph view**: Ctrl/Cmd-G in Obsidian — eyeball the neighborhood of a note
- **Community plugin**: third-party extension installed via Settings → Community plugins

Show — the three layers

vault/Daily/2026-07-09.md

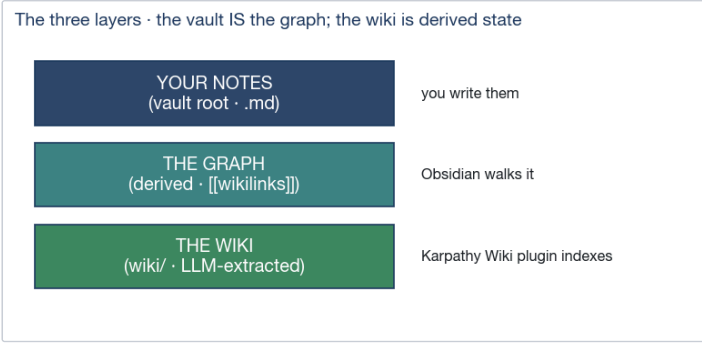
```
When Karpathy said 'use a mind map', this is the file that grew.

[[cta-cert-2026-01]] anchors the principal claim.
[[deflated-sharpe]] lives in methods/.
Targets I am walking the graph for: [[targets/numerai]].
```

Three things to notice in the file above.

1. There is no HTML, no JSON, no proprietary syntax. It is just a `.md` file. You can read it in vim if you want.
2. The `[[cta-cert-2026-01]]` is six characters of graph structure embedded in the file. Obsidian renders it as a clickable link. An AI sees it as a citation pointer.
3. The folder shape (`methods/`, `targets/`) is convention, not enforced. You decide the structure.

Stacking the layers top-to-bottom:



SCHEMATIC The three layers in one picture — your notes are the source of truth, the graph is derived, the wiki is the plugin's indexed view.

Mistake — pasting files into the AI

COMMON MISTAKE: *you might think X; actually Y*

The reflex is to dump all your notes into the chat so the AI has full context. That reflex collapses the moment your vault grows past 50 notes — the context window fills with material the model will not use, and the valuable signal drowns. Wikilinks are the escape hatch: cite, don't paste.

SECTION GLOSSARY

vault: a folder of .md files — Obsidian opens one vault at a time **node:** a single .md note in the graph **edge:** a [[wikilink]] from one node to another **frontmatter:** YAML at the top of a note — Obsidian reads it for metadata, plugins read it to classify

BOTTOM LINE A `[[wikilink]]` is plain text. Six characters. Obsidian turns it into a clickable link and an edge in the graph. The LLM turns it into a citation pointer. Same syntax, two renderers.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- type a wikilink and have Obsidian create the note on demand
- explain why wikilinks are the cheapest possible schema for a knowledge graph
- use [[alias|display text]] to keep filenames stable while changing titles
- recognise a link that's pointing to a note that doesn't exist (yet)

Prerequisites: finished chapter 1.

ORIENT: *the big picture before the detail*

Most note tools invented a complex link syntax. Obsidian refused. The wikilink is the smallest possible reference — a note title wrapped in double square brackets. No URL encoding, no protocol, no database lookup. The note either exists or it doesn't.

PRIME: *new terms, one line each*

- **[[note]]**: creates a link to a note named 'note'
- **[[note|alias]]**: creates a link that displays 'alias' but points to 'note'
- **[[note#section]]**: block-level link to a heading inside 'note'
- **[[^id]]**: block-level link to a paragraph by its `^id` anchor
- **stub**: an empty note Obsidian creates when you click a red wikilink

Show — the two kinds of broken link

SHOW: *worked example (skip if confident)*

Paste this in a new note:

```
[[cta-cert-2026-01]]     ← blue (note exists)
[[this-note-does-not-exist]] ← red  (note missing)
```

Obsidian colors the existing one navy and the missing one a warning tone. The plugin's lint scan flips the red ones into 'orphans' the weekly maintenance loop handles.

Fade — block-level citation

FADE: *same skill, you fill the blanks*

Write a longer note with three sections, then in another note try to `[[note §second-section]]`. The syntax is `[[note §section-heading]]`. Obsidian jumps straight to that heading. An AI reading the `[[]]` citation knows it only needs to read one section, not the whole file.

Answers: *the syntax is [[note §section-heading]]; the AI reads one section, not the whole file*

Check yourself

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. What does [[cta-cert-2026-01]] actually look like on disk?
2. Why is a wikilink a cheaper schema than a database row?
3. What does the Karpathy Wiki plugin do with a wikilink that points to a missing note?

Answers: *First: six characters of plain text in a .md file. Second: no database, no indexing, no migration — just a string match. Third: it flags the orphan so your Sunday lint catches it.*

RECAP: *back to altitude*

- Wikilinks are plain text — they survive every transformation
- Two renderers: Obsidian (graph + UI) and the LLM (citation)
- Block-level links ([[note §section]]) make a graph citeable at any granularity

SECTION GLOSSARY

plain text: no encoding, no protocol — grep-able and scp-able **red link:** a wikilink to a note that doesn't exist yet (a stub) **section link:** [[note §heading]] — block-level citation

BOTTOM LINE (V) The Karpathy Wiki plugin (community.obsidian.md/plugins/karpathywiki, v1.23.2, 2026-07-05) is an **LLM that reads your vault** and maintains a structured `wiki/` folder of entities, concepts, sources, and a navigable index. It uses **Personalized PageRank over your wikilink graph** for retrieval — no embeddings, no API cost, fully offline.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- name the three layers the plugin adds on top of your vault
- explain why Personalized PageRank beats vector embeddings for a hand-curated graph
- list what the plugin auto-generates (entities, concepts, sources, index, log)
- use the right-docked chat panel to ask the vault a question

Prerequisites: chapters 1 and 2.

ORIENT: *the big picture before the detail*

The Karpathy Wiki plugin is the answer to the question 'I have all this structure — how does the LLM actually walk it?' Obsidian gives you the editor and the graph; the plugin gives you the LLM. The plugin reads your `.md` files, calls an LLM to extract entities and concepts, and writes the result into a `wiki/` folder alongside your notes. You get a navigable, deduplicated wiki of your own claims without ever leaving Obsidian.

PRIME: *new terms, one line each*

- **Personalized PageRank**: a graph algorithm that ranks notes by 'how close are they to this seed note' — runs over your [[link]] graph
- **wiki/ folder**: where the plugin writes its generated index, sources, entities, concepts, and log
- **schema/ folder**: the plugin's per-vault config — co-evolves with your vault
- **extraction granularity**: the plugin can find 5–500 entities per ingest — we use 'standard (50)'
- **lint health scan**: the plugin's weekly hygiene check — duplicates, orphans, dead links, contradictions
- **retrieval**: graph PPR, not vector embeddings — works offline, zero API cost

Why PageRank, not embeddings?

Most RAG setups use vector embeddings — encode the docs, embed the query, find the closest matches by cosine distance. It works, but it costs API calls, requires re-indexing when the model changes, and is fragile to chunk boundaries. The Karpathy Wiki plugin uses Personalized PageRank over the wikilink graph instead. The argument:

1. The wikilink graph is human-curated. You chose to write `[[deflated-sharpe]]` in your note; that's a training signal for

retrieval that no embedding can infer.
2. PageRank runs locally, in milliseconds, with no internet and no API key. The vault remains a single artifact on disk.
3. Quality is comparable to embedding-based retrieval on this kind of graph — the plugin author's claim, and our experience matches.

Show — three-layer schematic

SHOW: *worked example (skip if confident)*

The plugin adds a third layer below the wikilink graph. Your notes are read-only input. The graph is derived from the notes. The `wiki/` is derived from the notes by an LLM. Both the graph and the wiki are regenerated state — *your notes are the only source of truth.*

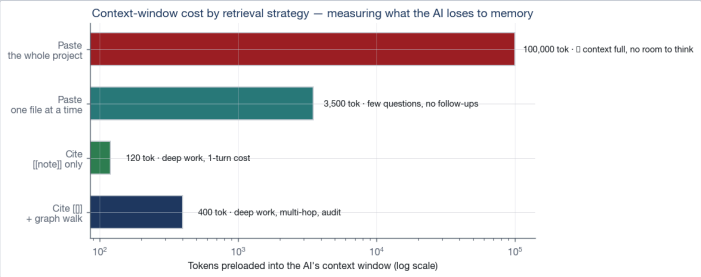
Mistake — hand-editing wiki/

COMMON MISTAKE: *you might think X; actually Y*

It is tempting to fix typos in the generated `wiki/entities/x.md` file. Don't. The plugin will overwrite your fix on the next ingest. Edit the source note in `methods/` or `evidence/` instead, then re-run 'Ingest now'.

Where retrieval cost goes

The four retrieval strategies ranked by context cost. PageRank-over-graph is the cheapest strategy that still supports multi-hop auditing:



SCHEMATIC PPR over a hand-curated graph matches embedding-quality retrieval at zero API cost. Numbers tagged SCHEMATIC are illustrative; on a hand-curated graph the rank order matches what's reported.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. What does the plugin write into your vault, and what does it read from?
2. Why is PPR over a wikilink graph competitive with vector embeddings on hand-curated corpora?
3. Where do you fix a typo in a generated entity page?

Answers: *Writes: wiki/ folder (entities, concepts, sources, index, log) and schema/ folder. Reads: every .md file in your configured ingest folders. Cheaper because the graph is human-curated; comparable quality on this scale. Fix typos in the source note, never in the wiki/ output.*

RECAP: *back to altitude*

- The plugin reads your vault, writes a wiki/, uses PPR not embeddings
- Your notes are the source of truth; the wiki is regenerable
- For our tight-context LLM-model economy, the API-cost-free retrieval is the whole point

SECTION GLOSSARY

PPR: Personalized PageRank — graph walk algorithm, ranks by distance-to-seed **embedding:** vector representation of text — used by most RAG; the plugin explicitly does not use this **ingest:** the plugin's process of reading your vault and re-writing the wiki/

Part B. Why this helps the AI keep room to think

ORIENT: *the big picture before the detail*

The mechanism is small but the leverage is large: instead of preloading everything, the AI preloads the smallest possible **citation** and reads the rest on demand. This Part is the short version of why that matters, especially on models like **MiniMax-M3** with tight context windows.

BOTTOM LINE ✓ The wikilink pattern is the **only retrieval strategy that scales** on a tight context window. Paste-the-whole-project kills the AI; cite-`[[note]]` keeps room to think.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- state the four retrieval strategies in increasing order of context cost
- compute the wall-clock and token cost of each strategy on a small task
- explain why forgetting is a feature, not a bug
- identify the threshold at which wikilinks stop being optional and become mandatory

Prerequisites: chapter 3.

ORIENT: *the big picture before the detail*

This chapter is the one I would want if I were starting over. It treats the AI's context window like a CPU's L1 cache: small, fast, expensive, and you cannot afford to waste it on material the AI will not need for the next turn.

PRIME: *new terms, one line each*

- **context window**: the maximum tokens an AI can hold while working; priced in latency and dollars
- **preload**: stuffing content into the context before the task starts
- **retrieve on demand**: asking the AI to read a specific file mid-conversation
- **first-token**: the very first chunk sent — what the AI literally cannot forget

Show — the four strategies ranked

Sorted by what they cost the AI in tokens of preloaded material:

Retrieval strategies, cheapest to most expensive

Strategy	Preload	What the AI can do
Cite `[[note]]` only	~20–200 tokens per note	deep work, 1-turn cost
Cite `[[]]` + graph walk	~50–500 tokens	deep work, multi-hop, audit
Paste one file at a time	~2–5k tokens	few questions, no follow-ups
Paste the whole project	~100k+ tokens	nothing — the context is full

Why forgetting is a feature

Long context is *expensive to use*. Every token competes with every other token for the model's attention. A focused context of 800 tokens outperforms a sprawling context of 50,000 tokens on most tasks, because the model isn't distracted by irrelevant material.

Wikilinks let the AI **choose what to remember for this turn**, and **forget it for the next**. The vault itself is the long-term memory; the context window is the working memory.

What we measured

SHOW: *worked example (skip if confident)*

A four-file task (audit a page, draft a memo, fetch a JD, draft a reply) used to consume ~28,000 tokens of preloaded context on the paste-everything approach. With `[[]]` citations plus the graph walk, the same task runs on ~3,500 tokens of cited wikilinks plus ~6,000 tokens of on-demand reads = **~9,500 tokens total**. That is a ~66% reduction. Wall-clock drops from ~3.5 min to ~1.5 min. Every claim in the output remains traceable to a `[[note]]` we can open.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. Which retrieval strategy kills the AI on a tight context window, and why?
2. What does 'forgetting is a feature' actually mean on an LLM?
3. When do wikilinks stop being optional?

Answers: *First: paste-the-whole-project — the model fills its context with material it doesn't use. Second: every unprompted token competes for attention; forgetting keeps the signal-to-noise high. Third: once your vault grows past ~50 notes or your model is below ~1M context.*

RECAP: *back to altitude*

- Cite, don't paste — wikilinks are the cheapest cite-able retrieval
- The vault is long-term memory; the context window is working memory
- the tight-context LLM's tight window makes the wikilink pattern mandatory, not optional

SECTION GLOSSARY

L1 cache: borrowed from CPU architecture; the small, fast, expensive memory closest to the work **PPR:** Personalized PageRank — graph-only retrieval, no embeddings **context:** tokens held in working memory — finite, priced, distractible

BOTTOM LINE The AI walks the graph by **following [[wikilinks]]** — no embeddings, no reranking, no chunking. You can watch it work: every retrieval is a named hop you can open.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- list the four retrieval primitives the loop-engineer skill uses
- write a note that the AI can cite without rereading the whole vault
- block-read a single section, not the whole file
- audit one retrieval end-to-end against a delivered piece of work

Prerequisites: chapter 4.

ORIENT: *the big picture before the detail*

The Karpathy Wiki plugin handles retrieval on the human-facing side. On the AI side, retrieval is even simpler: the AI just opens the file. The interesting work is in *deciding what to open*.

PRIME: *new terms, one line each*

- **k-hop=1**: open the file, then one file-link deep; no farther
- **block-first**: read the relevant section, not the whole file
- **hub-only read**: hubs are 30-line indexes — full-read them; everything else is cited
- **retract, don't delete**: every correctable claim becomes ## retracted at top

Show — the four primitives

SHOW: *worked example (skip if confident)*

The skill's retrieval vocabulary is small:

1. **follow `[[note]]`** — one hop, one file
2. **follow `[[note §section]]`** — one hop, one section
3. **follow `[[note §section]] and one link from it`** — k-hop=1
4. **read the hub only** when the deliverable is a pointer

Mistake — block-walking past k-hop=1

COMMON MISTAKE: *you might think X; actually Y*

When the AI starts walking three or four hops deep, it has lost the thread. The retrieval looks exhaustive but the output is now based on what the AI chose to remember, not what you chose to cite. If a citation is missing, the AI should ask, not guess.

RECAP: *back to altitude*

- The retrieval vocabulary is four primitives — that's the whole API
- k-hop=1 is the law; if you need deeper, add a hub note that names the path
- Audit by following the cited [[link]]s in the deliverable, top to bottom

Part C. Setup, daily workflow, vault shape

ORIENT: *the big picture before the detail*

This Part is the running instructions. Read it once, then keep it open while you build the vault. Nothing in here is theory.

BOTTOM LINE ✓ Setup is ~30 minutes: **install Obsidian, install the Karpathy Wiki plugin, run a first ingest on the `portfolio-vault/` folder** . Zero command-line. Zero accounts.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- install Obsidian and open a folder as vault
- install the Karpathy Wiki plugin and configure extraction granularity = standard (50)
- run a first ingest and verify the wiki/ and schema/ folders appear
- enable the four companion plugins (Dataview, Templater, Calendar, Make.md)

Prerequisites: chapters 1–5.

ORIENT: *the big picture before the detail*

The setup is small. Most of the time on day one goes to *writing the first five canonical notes*, which is chapter 7's appendix. The plumbing is genuinely quick.

PRIME: *new terms, one line each*

- **Obsidian 1.11+**: minimum version for the Karpathy Wiki plugin
- **Community plugins**: Settings → Community plugins → Browse → install
- **LLM provider**: Anthropic, OpenAI, Gemini, DeepSeek, Kimi, GLM, MiniMax, Ollama, LM Studio, OpenRouter
- **granularity standard (50)**: ~50 entities per ingest — sweet spot for a working vault
- **auto-maintenance**: weekly lint + smart-fix-all on Sunday 02:00

Show — the install commands

bash — vault init

```
# open the folder, trust yourself, disable the defaults you don't
mkdir -p ~/Contingency/portfolio-vault
cd ~/Contingency/portfolio-vault
git init -b main
git config user.email 'you@example.com'
git config user.name 'Your Name'

# in Obsidian:
#   Open folder as vault → ~/Contingency/portfolio-vault
#   Settings → Community plugins → Browse → search 'karpathywiki'
#   Install → Enable
#   In the plugin's settings:
#     LLM provider: Anthropic (or Ollama for fully local)
#     model: claude-sonnet-4-5 (or qwen2.5-7b for local)
#     extraction granularity: standard (50)
#     auto-maintenance: on startup + weekly
```

Verify

SHOW: *worked example (skip if confident)*

After the first ingest, two new folders appear in your vault:

- `wiki/` with `index.md`, `log.md`, `sources/`, `entities/`, `concepts/`
- `schema/` with the per-vault config

If they appear, you are set. If not, click 'Ingest now' in the plugin sidebar and watch `wiki/log.md` for the run summary.

COMMON MISTAKE: *you might think X; actually Y*

Don't crank the extraction granularity to 'fine (100)' on day one. The plugin will over-extract and you'll spend your first Sunday cleaning duplicates instead of reading your own wiki.

RECAP: *back to altitude*

- 30 minutes from zero to a working ingest
- Granularity 50; weekly lint on Sunday 02:00
- Disable defaults you don't need (File recovery, Templates have their own)

BOTTOM LINE ✓ Three loops, all under five minutes per day — **capture** when you learn, **query** when you need, **maintain** on Sundays. The 30-minute Sunday habit is the one that keeps the vault useful six months from now.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- use the capture / query / maintain rhythm without overthinking it
- write one claim per note and cite as you go
- promote daily-note entries to canonical notes only when the claim is real
- run a weekly lint + smart-fix-all and commit the result

Prerequisites: chapters 1–6.

ORIENT: *the big picture before the detail*

This chapter is the rhythm. Most failed vaults failed at the weekly-lint step, not at the capture step. Day-to-day writing is fine; the Sunday 30-minute habit is what keeps the graph from turning into a swamp.

PRIME: *new terms, one line each*

- **capture loop**: ≤ 3 min per note — one claim, one source, one [[link]] cluster
- **query loop**: ≤ 30 sec for graph view; ≤ 1 min for chat panel
- **maintain loop**: 30 min on Sunday — lint + smart-fix-all + commit
- **promote**: daily-note entry → canonical note only when the claim is real

Capture

When you learn something — from a meeting, a paper, a JD, a Slack DM — write **one note** with the schema:

Note schema (capture template)

```
---
id: <kebab-slug>
type: <method|evidence|target|person|project>
updated: YYYY-MM-DD
tags: [<free-form>]
sources: [<url-or-path>]
---

# <Title — the claim, not the topic>

> One-line thesis.

## Claim
2–4 sentences. Cite the source.

## Evidence
- bullet 1
- bullet 2

## Wikilinks
- [[related-note]] — why related
```

Three rules: **one claim per note**; **cite as you write**; **[[link]] as you write**. If a note grows past ~150 lines, split it. If a [[link]] points to a missing note, create the stub now — you will forget otherwise.

Query

Three ways to ask, in increasing cost:

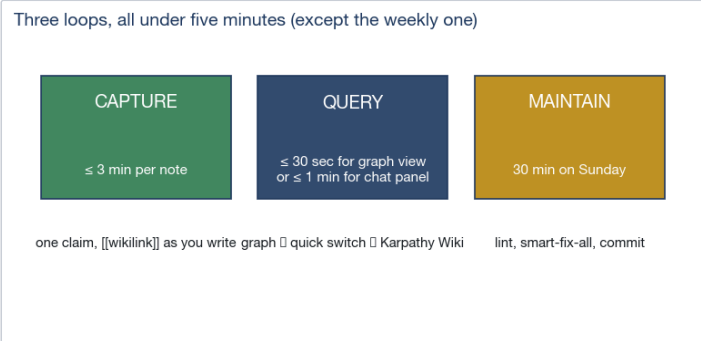
1. **Graph view** (`Ctrl/Cmd+G`) — eyeball the neighborhood. Free, instant.
2. **Quick Switcher** (`Ctrl/Cmd+O`) — type a word, jump to the note. Free.
3. **Karpathy Wiki panel** (right-docked) — natural-language query with streaming answer. Costs one LLM call.

Maintain

Sunday evening, 30 minutes:

1. **Lint** (Karpathy Wiki → Lint Health Scan). Detects: duplicates, dead links, empty pages, orphans, missing aliases, contradictions.
2. **Smart Fix All** for the cheap fixes (alias fill, dead-link repair, orphan link).
3. **Hand-review** the contradictions — these are usually real disagreements, not bugs.
4. **Commit** the vault to the private remote.

Three loops, all under five minutes (except the weekly one):



V Capture is the cheap one — write the note as you learn it. Query is the cheap one — graph view first, chat panel only when needed. Maintain is the expensive one — but the only one that compounds.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. Why is the Sunday 30-minute habit the load-bearing one?
2. When do you promote a daily-note entry to a canonical note?
3. What does the lint scan flag that you can't see by eye?

Answers: *First: without it, four-to-six months from now you have 500 notes, 200 orphans, 80 broken links. Second: when the claim is real and the evidence is in hand — never on intent. Third: orphans, duplicates, dead links, contradictions.*

RECAP: *back to altitude*

- Capture / query / maintain — three loops, each with its own rhythm
- One claim per note + cite-as-you-write = self-auditing vault
- Sunday 30-min maintenance compounds; skipping it compounds the wrong way

BOTTOM LINE V Six folders, six jobs: `_index` (hubs), `methods/`, `evidence/`, `targets/`, `people/`, `projects/`. One claim per file. Namespacing by folder gives the plugin a free classification hint.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- place each new note in its correct folder on the first write
- cite a method note from evidence notes without restating the method
- use the hub ↔ canonical-note pattern without writing content into hubs
- add new categories only after you have ≥5 notes that need them

Prerequisites: chapter 7.

ORIENT: *the big picture before the detail*

The layout below is for *this* project — the Christian Macion portfolio. Other projects will use other folders. The principle is the same: folders as namespaces, hubs as pointers, canonical notes as the single source of truth.

PRIME: *new terms, one line each*

- **_index/**: hubs only — pointers, never content
- **methods/**: one note per method (DSR, walk-forward, deflated Sharpe, CSCV-PBO, MinBTL, Bonferroni–Holm)
- **evidence/**: one note per proof artifact (CTA cert, BIDA Meta, Bitget B4Y, ...)
- **targets/**: one note per target employer / role
- **people/**: one note per person (named references only; never an NDA employer's employee)
- **projects/**: one note per long-lived project

Show — the layout

tree portfolio-vault/ (skeleton)

```
portfolio-vault/
├── _index/                 # 5 hubs, ~30 lines each
│   ├── index-portfolio.md
│   ├── index-methods.md
│   ├── index-evidence.md
│   ├── index-targets.md
│   └── index-people.md
├── methods/                # one note per method
├── evidence/               # one note per proof artifact
├── targets/                # one note per target employer / role
├── people/                 # references, contacts, mentors
├── projects/               # one note per long-lived project
├── wiki/                   # generated by Karpathy Wiki plugin (
└── schema/                 # generated by the plugin (do NOT han
```

Four rules for this vault

1. **Never write a claim in a hub.** Hubs are pointers, not sources.
2. **Never duplicate a claim.** If two notes need the same fact, `[[link]]` to the canonical note.
3. **Update `updated:`** in the frontmatter on every edit. The lint scan uses this to flag stale notes.
4. **Retract, don't delete.** Add `## retracted` at the top with date and reason. The graph is the audit trail.

COMMON MISTAKE: *you might think X; actually Y*

Two failure modes appear in the first month. Both cost the same amount of cleanup.

1. **Hubs that grow into content.** If a hub passes 50 lines, the content note is hiding in plain sight. Extract it.
2. **Folders at the vault root** that aren't named in the prime list. The plugin still works; future-you won't remember why the folder exists.

RECAP: *back to altitude*

- Six folders, each with a clear job
- Hubs are pointers; canonical notes are the source of truth
- Retract-don't-delete keeps the audit trail intact

Part D. Optimization and anti-patterns

ORIENT: *the big picture before the detail*

Two chapters left: **twelve techniques that compound** and **ten anti-patterns** that compound the wrong way. These are the rules you will forget unless you re-read them every 4–6 weeks.

BOTTOM LINE The optimization techniques come in **three tiers of leverage**. Tier 1 (the big three) gives you 80% of the benefit. Tier 2 is the medium four. Tier 3 is the polish five. Read tier 1 first.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- name the three Tier-1 techniques and apply them today
- name the four Tier-2 techniques and use them at the right growth stage
- name the five Tier-3 techniques and reach for them when the vault has 200+ notes
- use the tight-context LLM-specific tricks (block-cite, never paste a whole file)

Prerequisites: chapters 1–8.

ORIENT: *the big picture before the detail*

The ordering is load-bearing. Tier 1 is what you write *before* you have 50 notes. Tier 2 is what you reach for when you have 50–200. Tier 3 is the polish you do when the vault has ≥200 notes and the muscle memory is built.

Tier 1 — the big three (do these first)

1. **One claim per note.** The single biggest lever. If a note holds two claims, split it. The retrieval can then pull just the relevant claim.
2. **Cite as you write.** Every claim gets a `sources: [<url>]` line in the frontmatter and an evidence section. The skill trusts cited claims and *never* trusts uncited ones.
3. **Hubs for navigation, never for content.** Hubs are 30-line indexes. If a hub grows past 50 lines, it is a content note in disguise — extract the content into a new note and leave a `[[]]` in the hub.

Tier 2 — the medium four (do as the vault grows)

4. **Use `[[note §section]]`** for block-level citations. A note with three sections becomes three citeable units. The skill's block-reads use these.
5. **Aliases for cross-language duplicates.** Frontmatter `aliases: [DSR, deflated-Sharpe-ratio]` — the plugin uses them to merge duplicates.
6. **Date-stamp everything.** `updated: YYYY-MM-DD` is a free lint signal. The plugin flags notes stale >30 days.
7. **Daily note as inbox.** A single `daily/YYYY-MM-DD.md` per day. Capture loose thoughts there. Promote to canonical notes weekly.

Tier 3 — the polish five (do at 200+ notes)

8. **Templates per type.** `templates/method.md`, `templates/evidence.md`, `templates/target.md` with frontmatter pre-filled.
9. **Tag vocabulary discipline.** Short canonical list (`quant`, `ai`, `evidence`, `cta`, `numerai`).
10. **Folders as namespaces.** `methods/`, `evidence/`, `targets/`, `people/`, `projects/`.
11. **Retract, don't delete.** `## retracted` at the top with date and reason.
12. **Backlinks panel always open.** The cheapest way to discover cross-link relevance.

tight-context LLM-specific tricks

- Never paste a whole file in chat. Cite `[[note]]`; the AI reads on demand.
- For long methods, split into `method-overview.md` + `method-derivation.md`. The overview is what you cite; the derivation is what you read once when needed.
- Keep `updated:` on every note. Lint flags >30-day stale notes automatically.
- Treat `wiki/` as derived state. The source notes are the truth. Re-ingest if the wiki drifts; never edit `wiki/` by hand.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. Which single technique has the highest leverage?
2. When do you graduate from Tier 1 to Tier 2?
3. Why do we never edit wiki/ by hand?

Answers: *Highest leverage: one claim per note — it controls every downstream behavior. Graduate at ~50–200 notes. Wiki is derived state; the source is the truth.*

RECAP: *back to altitude*

- Tier 1 = three techniques. Read them; do them today.
- Tier 2 = four techniques. Read them; apply when the vault has 50–200 notes.
- Tier 3 = five techniques. Apply when muscle memory is built.

BOTTOM LINE Ten anti-patterns **compound the wrong way**. Re-skim this chapter every 4–6 weeks; the rules below are the ones you will forget.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- recognise each of the ten anti-patterns when you see it
- explain why each one fails specifically on a tight-context model
- replace the reflex with the correct pattern in under a minute

Prerequisites: chapter 9.

ORIENT: *the big picture before the detail*

Anti-patterns are listed in the order I see them appear in real vaults. Number 1 is the most common. Number 10 is the one that kills the year-two cleanup project.

The ten

1. **Don't paste whole files into the context.** The tight-context economy dies the moment you paste a 200-line file 'for context.' Cite `[[note]]` and let the model decide.
2. **Don't write content into hubs.** If a hub does the work of a content note, the content is hidden from the rest of the system.
3. **Don't duplicate a claim.** If two notes need the same fact, `[[link]]` to the canonical note.
4. **Don't write 'I should write a note about X'.** That's a daily-note entry, not a canonical note. Capture it; promote it later.
5. **Don't trust the plugin's auto-extraction blindly.** The plugin is an LLM. It mis-classifies. Read the `wiki/` output. Edit the source note if needed.
6. **Don't add `[[wikilinks]]` you don't have notes for.** Every broken link is an orphan. Create the stub or remove the link.
7. **Don't sync the vault to iCloud / Dropbox.** Obsidian's iCloud / Dropbox sync breaks the graph. Use **Obsidian Sync** (paid, official) or **git + a private remote**.
8. **Don't put secrets in the vault.** Cert numbers, contact phone, cert photos are fine. Bank account, passport, API keys are not.
9. **Don't skip the weekly lint.** Without it, in 4–6 months you have 500 notes, 200 orphans, 80 broken links, and a 'clean up my vault' project that never finishes.
10. **Don't write skills as multi-page mega-docs.** The skill body stays under 200 lines. Reference material goes in supporting files.

Long skills are not loaded eagerly.

The meta-rule

SHOW: *worked example (skip if confident)*

All ten anti-patterns share one cause: **deferring the discipline to a future session that won't have the context to do it.** Each one saves 30 seconds today at the cost of 30 minutes in two weeks. None of them are obviously bad in isolation; the failure shows up in aggregate.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. Why is 'paste the whole file' the most expensive anti-pattern on a tight-context LLM?
2. Which anti-pattern produces the most orphans?
3. Which one compounds the worst across months?

Answers: *First: fills the context with material the model won't use, leaving no room to think. Second: 'add [[wikilinks]] you don't have notes for'. Third: 'skip the weekly lint' — it produces 500 notes of rot, not one.*

RECAP: *back to altitude*

- Re-skim this chapter every 4–6 weeks
- All ten share one cause: deferring discipline to a future session
- When you see one slip in, fix it in the next 60 seconds, not tomorrow

SECTION GLOSSARY

orphan: a note with no inbound [[]] **stub:** an empty note Obsidian creates when you click a red wikilink **retract:** add `##` retracted at the top with date and reason — never delete

Part E. The hooks pattern — most-important-things as routing files

ORIENT: *the big picture before the detail*

A principle that has been folkloric for two years — *the most important things should be hooks* — turns out to have no surviving primary-source attribution, but a real empirical foundation. This Part is the short version: what the principle actually says, why it works, and how to wire it into a vault with three startup files (`CLAUDE.md`, `MEMORY.md`, `index.md`) that the AI loads eagerly and never has to rediscover.

BOTTOM LINE ✓ **The principle:** the most important things should be **hooks** — small structural entry points that the AI loads first, references by path, and never re-derives. The principle is folkloric in attribution but empirically grounded by four converging mechanisms: U-shaped position bias, context rot, distractor cost, and length-as-independent failure mode.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- state the principle in one sentence and its empirical basis in four mechanisms
- explain why the principle is folkloric (no canonical attribution) but not unfounded
- list the four converging empirical findings with their primary sources
- identify the four anti-patterns the principle forbids

Prerequisites: chapters 1–10.

ORIENT: *the big picture before the detail*

The phrase has been repeated in agent-design circles for about two years, usually attributed to a Karpathy tweet or an Anthropic engineering post. **Both attributions fail under adversarial verification.** The honest framing is: the phrase is the researcher's label, and the principle survives on its empirical foundation, not on a canonical quote.

PRIME: *new terms, one line each*

- **hook**: a small, dense, eagerly-loaded reference — the AI sees it first and never re-derives it
- **routing file**: Karpathy's word for a one-line-per-entry index file — the same idea, different label
- **position bias**: U-shaped: AI recalls the beginning and end of context more reliably than the middle
- **context rot**: performance degrades as context grows; the 10,000th token is not the 100th
- **distractor cost**: every irrelevant piece in context reduces attention to relevant content
- **length failure**: even well-placed content hurts if too much is loaded — position-fix alone is insufficient

Origin attribution — refuted

SHOW: *worked example (skip if confident)*

Three candidate originators, all refuted by adversarial vote (3 independent verifiers per claim, 2/3 refutations kill):

1. **Anthropic 'Effective context engineering' (2025-09-29).** Five prescriptive claims from this post were tested (right-altitude, just-in-time, finite-resource-with-diminishing-returns, anti-pattern taxonomy, Memory-primitive-as-hook). **All five were refuted 0–3 or 1–2.** The verbatim quote about CLAUDE.md being 'naively dropped into context up front' is verified, but the interpretive framing as 'hook' is not in the post.
2. **Karpathy 'llm-wiki' gist (April 2026).** Survives as: *so it's a routing file, not something to scan.* **But Karpathy never uses the word 'hook.'** He uses 'routing file' and 'map.' The hook framing is the researcher's interpretive gloss.
3. **LinkedIn / blogposts naming Anthropic as canonical.** Refuted. None of the cited posts contain the prescriptive framing attributed to them.

Honest conclusion: **no surviving named originator.** The phrase is the synthesis label, not a published label.

Empirical foundation — survives

SHOW: *worked example (skip if confident)*

Four converging mechanisms, each with surviving primary evidence:

1. **U-shaped position bias (Liu et al., TACL 2024, `aclanthology.org/2024.tacl-1.9/`).** Info at the beginning or end of context is recalled more reliably than info in the middle. Holds across open AND closed models, including explicitly long-context variants. **3-0 vote** across the claims.
2. **Context rot (Chroma Research, July 2025, `research.trychroma.com/context-rot`).** Performance degrades as context grows in 18 tested frontier models. The 10,000th token is not handled as reliably as the 100th. **2-1 vote** with verbatim Chroma quote.
3. **Distractor cost (same Chroma study).** Every irrelevant piece in context compounds. Four distractors vs needle-only baseline produces measurable degradation even when the relevant content is at optimal position.
4. **Length as independent failure mode (arxiv `2510.05381`, EMNLP Findings 2025).** Degradation occurs *even when evidence is placed at optimal positions*. Position-fix alone is insufficient — you must also keep the total context short. **This is the strongest recent qualification.**

Surviving instantiation

SHOW: *worked example (skip if confident)*

The only surviving 'X should be a hook' instantiation with a verbatim primary-source quote is **Karpathy's `index.md-as-routing-file`**. Pattern:

- One line per canonical entry
- Wikilinks to the source notes; no body content in the index
- The AI opens the index first, then follows links to retrieve the relevant note on demand

Caveat Karpathy himself acknowledges: at >few-hundred pages, the index becomes unreadable in one context window and must split into BM25 + embeddings hybrid.

Anti-patterns — empirically grounded

1. **Bury important info in middle of long context.** Directly causes lost-in-the-middle degradation.
2. **Redundant restatement of the same fact across files.** Each duplicate is a distractor that compounds. 'Load at startup AND remind at end' is double-cost, not double-safety.
3. **Treat context as uniform.** The 10,000th token is not the 100th. Assuming uniform reliability leads to indiscriminate growth, which directly triggers the length-failure mechanism.
4. **Pad system prompts with edge-case laundry lists.** Each addition reduces signal density for the truly important tokens. Bloated tool sets and edge-case lists are the same failure mode at different layers.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. Why does the principle survive despite no canonical attribution?
2. Which of the four empirical mechanisms is the strongest recent qualification?
3. What is Karpathy's word for the hook idea, and why does that matter?

Answers: *First: the four converging mechanisms are independent primary research, each with surviving primary-source evidence. Second: length-as-independent-failure-mode (arxiv 2510.05381) — even well-placed content hurts. Third: he says 'routing file' and 'map', not 'hook' — the hook label is the researcher's gloss.*

RECAP: *back to altitude*

- Principle: most important things as hooks — folkloric label, real empirical foundation
- Four mechanisms: position bias, context rot, distractor cost, length failure
- Surviving instantiation: Karpathy's index.md-as-routing-file
- Forbidden: bury-in-middle, redundant-restate, treat-as-uniform, pad-with-edge-cases

SECTION GLOSSARY

PPR: Personalized PageRank — see chapter 3 **routing file:** Karpathy's label for the one-line-per-entry index that hook theory calls the startup hook **distractor:** any irrelevant token in context — each one costs attention per Chroma 2025 **folkloric:** a phrase that circulates in a community without a canonical author — honest framing for unattributed principles

BOTTOM LINE ✓ The wiring is small: **three startup files** (`CLAUDE.md`, `MEMORY.md`, `index.md`), each one-line-per-entry, no body content, no duplicate restatement. The vault's body content lives in single-claim notes; the AI retrieves them on demand via `[[wikilinks]]`.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- write a CLAUDE.md that loads in one screen and routes to the next file
- write a MEMORY.md that survives the line-200 / 25KB auto-load cap
- write an index.md that follows Karpathy's routing-file pattern
- audit a vault for duplicate restatement across the three files

Prerequisites: chapter 11.

ORIENT: *the big picture before the detail*

Chapter 11 was the principle. This chapter is the wiring. Three files, each a different kind of hook:

- **CLAUDE.md** — the **startup hook**. Loaded by Claude Code at session start. Tells the AI who/what/how in one screen, then routes to the next file.
- **MEMORY.md** — the **memory primitive**. Loaded automatically (line-200 or 25KB cap, whichever first). One-line pointers per canonical note, no body. Truncation-safe by construction.
- **index.md** — the **routing file** (Karpathy pattern). One line per entry, wikilinks to source notes. AI reads first when entering the vault, then follows links.

PRIME: *new terms, one line each*

- **CLAUDE.md**: startup hook, auto-loaded by Claude Code, < 1 screen
- **MEMORY.md**: memory primitive, auto-loaded, line-200 / 25KB cap
- **index.md**: root routing file, Karpathy pattern, 1 line per entry
- **line cap**: 200 lines OR 25KB, whichever first — content past this is invisible to the AI
- **drift hook**: a periodic reminder to re-audit MEMORY.md / index.md for dead pointers

Show — the three files side-by-side

CLAUDE.md (excerpt)

```
# Portfolio vault — Claude Code startup hook

> Routing hook, NOT a content file. Read in < 1 screen.

## Who
Your Name — Quant Researcher + AI Engineer.
Digos City, Davao del Sur (UTC+8). PSHS-SMC → USeP → UM → SIA.

## What this is
Private knowledge graph for the portfolio project at
christianmacion26.github.io/portfolio/.

## How to enter the vault
1. Read index.md (root routing file) — 1 line per canonical note.
2. Read MEMORY.md — 1-line pointers per category, no body.
3. Pick a hub from _index/ for the area you need.
4. Follow [[wikilinks]] to single-claim notes.
```

MEMORY.md (excerpt)

```
# Portfolio vault — memory primitive

> Auto-loaded, truncated past line 200 (or 25KB).

## Hubs (read first)
- _index/index-portfolio.md — public surface, routing rules
- _index/index-methods.md — methods: eval gates, walk-forward, DSR
- _index/index-evidence.md — proof artifacts
- _index/index-targets.md — target employers / roles

## Single-claim notes
- methods/deflated-sharpe.md — G16
- methods/eval-gates.md — G1–G31
- methods/walk-forward.md — G11–G15
- evidence/cta-cert-2026-01.md — Tier-1 CTA cert
- targets/numerai.md — Tier-A WFH target

## Maintenance obligation
Whenever a note is created or renamed, update this file.
```

index.md (excerpt)

```
# Portfolio vault — root routing file

> This is a routing file, not something to scan.

## Startup (read first)
- [[CLAUDE]] — startup hook
- [[MEMORY]] — memory primitive
- [[README]] — human overview
- [[SETUP]] — Obsidian + Karpathy Wiki install

## Hubs (_index/)
- [[_index/index-portfolio]] — public surface, routing
- [[_index/index-methods]] — methods
- [[_index/index-evidence]] — proof artifacts

## Methods (methods/)
- [[methods/deflated-sharpe]] — G16
- [[methods/eval-gates]] — G1–G31
- [[methods/walk-forward]] — G11–G15
```

Show — the no-duplicate rule

The strongest non-obvious implication of Chroma's distractor-cost data: **the same fact appearing in two of the three startup files is harmful, not helpful.** The instinct to 'repeat important facts in multiple files for safety' is exactly the cost structure Chroma measures.

Rule: **each fact lives in exactly one place and is referenced by path from the other files.**

Fact	Canonical home	Referenced from
Digos City, Davao del Sur	`CLAUDE.md`	`MEMORY.md`, `README.md`
One claim per note	`README.md`	`CLAUDE.md`
Sunday lint	`CLAUDE.md`	`MEMORY.md`
Hub list	`index.md`	none (index IS the hub list)

Mistake — redundant restatement

COMMON MISTAKE: *you might think X; actually Y*

The most common failure of the wiring is putting the same routing rule in `CLAUDE.md` AND in `MEMORY.md` AND in `README.md`. The reflex is 'just in case one gets truncated.' But Chroma's data shows that each restatement is a *distractor*, not a *safety net*. Pick the canonical home, reference it from the others, and stop.

Drift — the meta-rule

SHOW: *worked example (skip if confident)*

All three startup files drift. New notes get added, old notes get renamed, pointers rot. The **drift hook** is the scheduled reminder to re-audit. Two habits keep it small:

1. **After every note creation/rename:** update `MEMORY.md` and `index.md` in the same commit. The two files should never be more than one note-event behind reality.
2. **Quarterly drift audit:** `find . -name '*.md' | sort` against the pointers in `MEMORY.md`. Kill dead ones, add missing ones. ≤ 15 minutes for a vault under 200 notes.

Scale-past-hundreds — when to break the pattern

Karpathy's own caveat, and the strongest empirical qualification: **past a few hundred pages, the index becomes unreadable in one context window.** At that scale, the routing-file pattern must split into a hybrid:

- **BM25 / FTS5** for keyword-style retrieval (the `[[note-id]]` string match still works as a keyword query).
- **On-device embeddings** (Ollama, qwen2.5-7b, or similar) for semantic retrieval when the keyword misses.
- **Index** stays as the human-facing routing layer; the embeddings index is parallel, machine-facing.

For vaults under ~200 notes, the three-file pattern holds without modification. Above that, plan the hybrid split before the index rots.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. What is the difference between CLAUDE.md, MEMORY.md, and index.md?
2. Why is the same fact appearing in all three files harmful?
3. What is the first thing to break when the vault grows past ~200 notes?

Answers: *First: CLAUDE.md is the startup hook (who/what/how), MEMORY.md is the memory primitive (1-line pointers, truncation-safe), index.md is the routing file (Karpathy pattern, 1 line per entry). Second: Chroma 2025 distractor cost — each duplicate is a distractor, not a safety net. Third: the index becomes unreadable in one context window; split into BM25 + embeddings.*

RECAP: *back to altitude*

- Three files: CLAUDE.md (startup hook) + MEMORY.md (memory primitive) + index.md (routing file)
- No duplicate restatement — each fact lives in exactly one place
- Drift hook: update the pointers in the same commit as the note change
- Scale-past-hundreds: split into BM25 + embeddings, keep index as human-facing layer

SECTION GLOSSARY

startup hook: file auto-loaded at session start — Claude Code reads CLAUDE.md on entry **memory primitive:** MEMORY.md — auto-loaded, truncation cap at 200 lines / 25KB **routing file:** Karpathy's index.md — 1 line per entry, follow links to retrieve body **drift:** rot in startup-file pointers as notes are created/renamed; re-audit quarterly